

MongoDB Practice Questions

Covers - CRUD, operators, aggregation, joins, indexes, ranking. 80 practical questions plus interview theory.

Run the sample data below first. Every question uses these three collections. Solutions are in the separate answer document.

Sample Data

students (database: school)

```
use school
db.students.insertMany([
  { name:'Aarav', city:'Delhi', subject:'Math', marks:85 },
  { name:'Neha', city:'Delhi', subject:'Science', marks:67 },
  { name:'Rohan', city:'Delhi', subject:'Math', marks:90 },
  { name:'Priya', city:'Mumbai', subject:'Science', marks:78 },
  { name:'Karan', city:'Mumbai', subject:'Math', marks:97 },
  { name:'Sneha', city:'Mumbai', subject:'Science', marks:88 },
  { name:'Vikram', city:'Bangalore', subject:'Math', marks:69 },
  { name:'Divya', city:'Bangalore', subject:'Science', marks:93 },
  { name:'Anita', city:'Chennai', subject:'English', marks:72 },
  { name:'Rahul', city:'Chennai', subject:'Math', marks:80 },
  { name:'Meera', city:'Pune', subject:'Science', marks:91 },
  { name:'Sanjay', city:'Pune', subject:'Math', marks:84 },
  { name:'Pooja', city:'Kolkata', subject:'English', marks:76 },
  { name:'Tarun', city:'Kolkata', subject:'Science', marks:95 },
  { name:'Riya', city:'Noida', subject:'Math', marks:89 }
])
```

courses (database: school)

```
db.courses.insertMany([
  { studentName:'Aarav', course:'Web Development', duration:'6 months', fees:15000 },
  { studentName:'Rohan', course:'Data Science', duration:'8 months', fees:25000 },
  { studentName:'Karan', course:'Machine Learning', duration:'10 months', fees:30000 },
  { studentName:'Divya', course:'Web Development', duration:'6 months', fees:15000 },
  { studentName:'Meera', course:'Cloud Computing', duration:'5 months', fees:20000 },
  { studentName:'Tarun', course:'Data Science', duration:'8 months', fees:25000 },
  { studentName:'Riya', course:'App Development', duration:'7 months', fees:18000 },
  { studentName:'Priya', course:'Web Development', duration:'6 months', fees:15000 }
])
```

employees (database: company)

```
use company
db.employees.insertMany([
  { name:'Aarav Sharma', age:28, city:'Delhi', role:'Developer', salary:75000, department:'Engineering' },
  { name:'Priya Patel', age:34, city:'Mumbai', role:'Manager', salary:95000, department:'Engineering' },
])
```

```

    { name:'Rahul Kumar',    age:25, city:'Delhi',    role:'Developer', salary:60000,
    department:'Engineering' },
    { name:'Sneha Reddy',    age:30, city:'Bangalore', role:'Designer',  salary:70000,
    department:'Marketing' },
    { name:'Vikram Singh',   age:42, city:'Mumbai',    role:'Manager',   salary:110000,
    department:'Sales' },
    { name:'Ananya Iyer',    age:27, city:'Pune',      role:'Analyst',   salary:65000,
    department:'Finance' },
    { name:'Karan Mehta',    age:38, city:'Delhi',    role:'Lead',      salary:105000,
    department:'Engineering' },
    { name:'Divya Nair',     age:29, city:'Bangalore', role:'Developer',  salary:72000,
    department:'Engineering' },
    { name:'Arjun Verma',    age:45, city:'Chennai',   role:'Director',   salary:150000,
    department:'Sales' },
    { name:'Pooja Joshi',    age:31, city:'Pune',      role:'Analyst',   salary:68000,
    department:'Finance' },
    { name:'Rohan Gupta',    age:26, city:'Delhi',    role:'Designer',   salary:58000,
    department:'Marketing' },
    { name:'Meera Kapoor',   age:36, city:'Mumbai',    role:'Manager',   salary:98000,
    department:'HR' },
    { name:'Sanjay Pillai',  age:40, city:'Chennai',   role:'Lead',      salary:102000,
    department:'Sales' },
    { name:'Neha Bhatt',     age:24, city:'Pune',      role:'Intern',     salary:35000,
    department:'HR' },
    { name:'Tarun Saxena',   age:33, city:'Bangalore', role:'Developer',  salary:78000,
    department:'Marketing' }
  ])
})

```

Easy (Questions 1 to 25)

1. Insert one new student: name 'Aditya', city 'Delhi', subject 'Math', marks 88.
2. Insert two students in a single command: Kavya (Mumbai, Science, 81) and Manish (Pune, Math, 74).
3. Find all students in the collection.
4. Find all students from the city Delhi.
5. Find a single student whose name is 'Karan'.
6. Find all students with marks greater than 80.
7. Find all students with marks less than or equal to 70.
8. Find students from Mumbai whose subject is Science.
9. Find all students but show only the name and marks fields, hiding the `_id`.
10. Find all students sorted by marks in ascending order.
11. Find all students sorted by marks in descending order.
12. Show only the top 3 students by marks.
13. Skip the first 5 students and return the rest.
14. Count how many students are in the collection.
15. List the distinct cities present in the students collection.
16. Update Neha's marks to 70.

17. Add a field 'zone' set to 'North' for every student in Delhi.
18. Delete the student named 'Vikram'.
19. Find students whose subject is either Math or English.
20. Find all students whose city is NOT Delhi.
21. Find all employees with a salary greater than 90000.
22. Find all employees in the Engineering department.
23. Find the top 5 highest-paid employees.
24. Show only the name and salary of all employees, hiding the _id.
25. Find all employees whose role is 'Manager'.

Medium (Questions 26 to 60)

26. Find students whose marks are between 70 and 90 inclusive.
27. Find students who are in Delhi or Mumbai, using a single list operator.
28. Find students who are NOT in Delhi or Mumbai, using a single list operator.
29. Find students whose name starts with the letter 'R'.
30. Find students whose name ends with the letter 'a'.
31. Find students whose name contains 'ar', ignoring case.
32. Using a logical operator, find students in Delhi whose marks are at least 85.
33. Using a logical operator, find students whose marks are above 95 or below 70.
34. Find students whose marks are NOT greater than 80 (use a negation operator).
35. Find students who are neither in Delhi nor have marks equal to 90.
36. Find students where the 'marks' field exists.
37. Find employees whose salary is stored as an integer type.
38. Count how many students there are in each city.
39. Find the average marks for each city.
40. Find the total salary paid out in each department.
41. Find the highest salary in each department.
42. Find the lowest marks for each subject.
43. Count how many employees are in each department.
44. Find the average marks per city, but only for students who scored above 80.
45. Return all students with the field 'name' renamed to 'studentName' and 'marks' renamed to 'score'.
46. Return each student with a new field 'bonusMarks' equal to marks plus 5.
47. Add a field 'grade' that is 'A' if marks are 90 or above, otherwise 'B'.
48. Add a field 'grade': 'A' for 90+, 'B' for 80 to 89, otherwise 'C'.
49. Using the aggregation pipeline, find the top 3 scorers.

50. Using the aggregation pipeline, find the 2 lowest scorers.
51. Using the aggregation pipeline, find the 3rd and 4th highest scorers.
52. Show page 2 of employees with 5 employees per page (sorted by salary descending).
53. Count how many employees earn more than 70000, using a count stage.
54. Group employees by city; show average salary and headcount, sorted by average salary descending.
55. Find employees who are in Engineering or earn more than 100000.
56. Find students whose marks are one of 85, 90, or 95.
57. Group students by subject and produce a list of student names in each subject.
58. Find the average salary by department, sort it, and output only department and average salary cleanly.
59. Count how many employees hold each role.
60. Find students whose subject is Math and whose marks are at least 80.

Hard (Questions 61 to 80)

61. Join the students collection with courses so each student shows their matching course(s) in an array.
62. Join and flatten: show each student with their course as a flat field, dropping students who have no course.
63. Do the same join and flatten, but KEEP students who have no course.
64. After joining students with courses, find the total fees collected per city.
65. Find the average marks for each combination of city and subject (multi-field grouping).
66. In multiple stages: first find the subject-wise average per city, then the overall average per city, sorted best first.
67. Build a full pipeline: join with courses, flatten, add a grade (A/B/C/D), group by city with average/max/min marks and a count plus the list of students, sort by average marks, format the output, and save it to a collection called 'finalReport'.
68. Build a city report containing student count, average marks, and total course fees per city, and save it to 'cityReport'.
69. Find the top 3 scorers including ties (everyone in the top 3 distinct scores).
70. Assign every student a unique sequence number ordered by marks descending.
71. Rank students by marks using RANK semantics (tied students share a rank, then a gap follows).
72. Rank students within each city separately, and return the top scorer of every city.
73. Find the top 2 earners in each department.
74. Create an index on the employees department field, then explain a department query. What changes in the plan?
75. Run explain('executionStats') on a salary query before any index exists. What stage is used and what is totalDocsExamined?

- 76.** Create a compound index on students of city ascending and marks descending. Write one query that can use it and one that cannot, and say why.
- 77.** Create a unique index on the students name field, then attempt to insert a duplicate name and describe what happens.
- 78.** Create a text index on the employees name and city fields, then search for 'Delhi'.
- 79.** List all indexes on the employees collection, then drop the department index.
- 80.** A query filters employees by city and sorts the result by salary. Which single index would make it fast, and why? Give the createIndex command.

Interview Theory Questions

Conceptual questions covering the whole course. Answer in your own words.

- 1.** What is MongoDB, and how does it differ from a relational (SQL) database?
- 2.** What is BSON, and how is it different from JSON?
- 3.** Explain the hierarchy: database, collection, document, field.
- 4.** What is the `_id` field, and who generates it if you don't provide one?
- 5.** Map the main SQL terms (table, row, column, join) to their MongoDB equivalents.
- 6.** When would you choose MongoDB over a SQL database, and when would you not?
- 7.** What is the difference between `insertOne` and `insertMany`?
- 8.** What is the difference between `updateOne`, `updateMany`, and `replaceOne`?
- 9.** What is an upsert, and how do you trigger one?
- 10.** What is the difference between a null value and a missing field, and how do you query each precisely?
- 11.** What is projection? How do you include versus exclude fields, and can you mix 1 and 0?
- 12.** What is the difference between an implicit AND and an explicit `$and`?
- 13.** What is the difference between `$in` and `$all`?
- 14.** Why do you need `$elemMatch` when querying an array of objects?
- 15.** What does `$expr` allow you to do, and why must field names carry a `$` prefix inside it?
- 16.** What is the aggregation pipeline, and how does it differ from `find()`?
- 17.** Why does the order of pipeline stages matter, and why is `$match` usually placed early?
- 18.** What is the difference between `$sum:1` and `$sum:'$field'`?
- 19.** What is the difference between the `$count` stage and a `$sum:1` accumulator?
- 20.** What is the difference between `$add`, `$sum`, and `$addFields`?
- 21.** What is `$lookup`? Explain `from`, `localField`, `foreignField`, and `as`.
- 22.** Why is `$unwind` usually used right after `$lookup`?
- 23.** What happens to documents with no match after `$unwind`, and how do you keep them?
- 24.** What does `$out` do, and what are its constraints?

- 25.** What is an index, and how does it improve query performance?
- 26.** What is the difference between COLLSCAN and IXSCAN in an explain plan?
- 27.** What does totalDocsExamined tell you about a query?
- 28.** What is a compound index, and what is the prefix rule?
- 29.** What is a unique index, and what happens if duplicate data already exists?
- 30.** What is a text index, and which operator queries it?
- 31.** What are the tradeoffs of indexes — why not index every field?
- 32.** Explain RANK, DENSE_RANK, and ROW_NUMBER, and name their MongoDB equivalents.
- 33.** Why does sort().limit() not reliably give the 'top N' when there are ties?
- 34.** What is schema validation, and how does \$jsonSchema enforce it?
- 35.** Embedding versus referencing: when would you use each in data modelling?